



THE UNIVERSITY OF
MELBOURNE

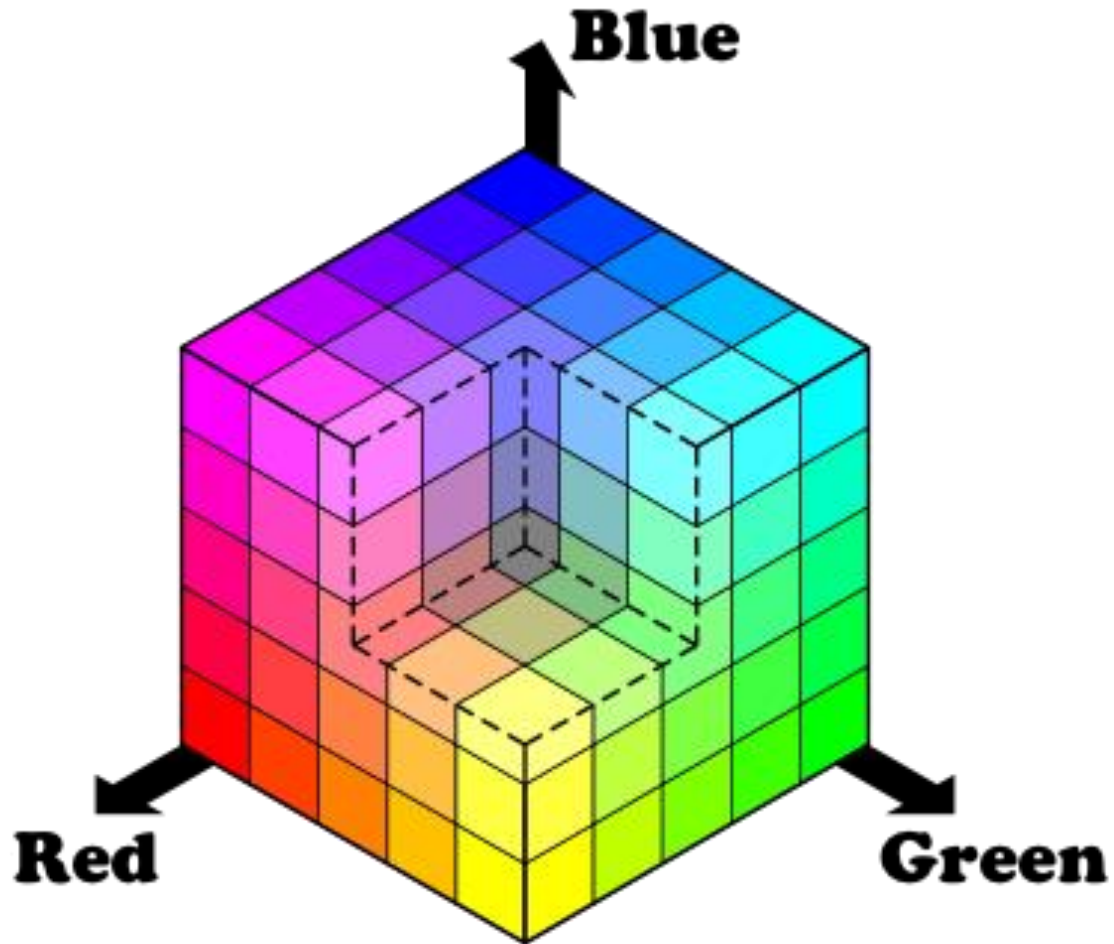
Computer Vision COMP90086 Workshop Week 4

Thi Xuan May Le (May)

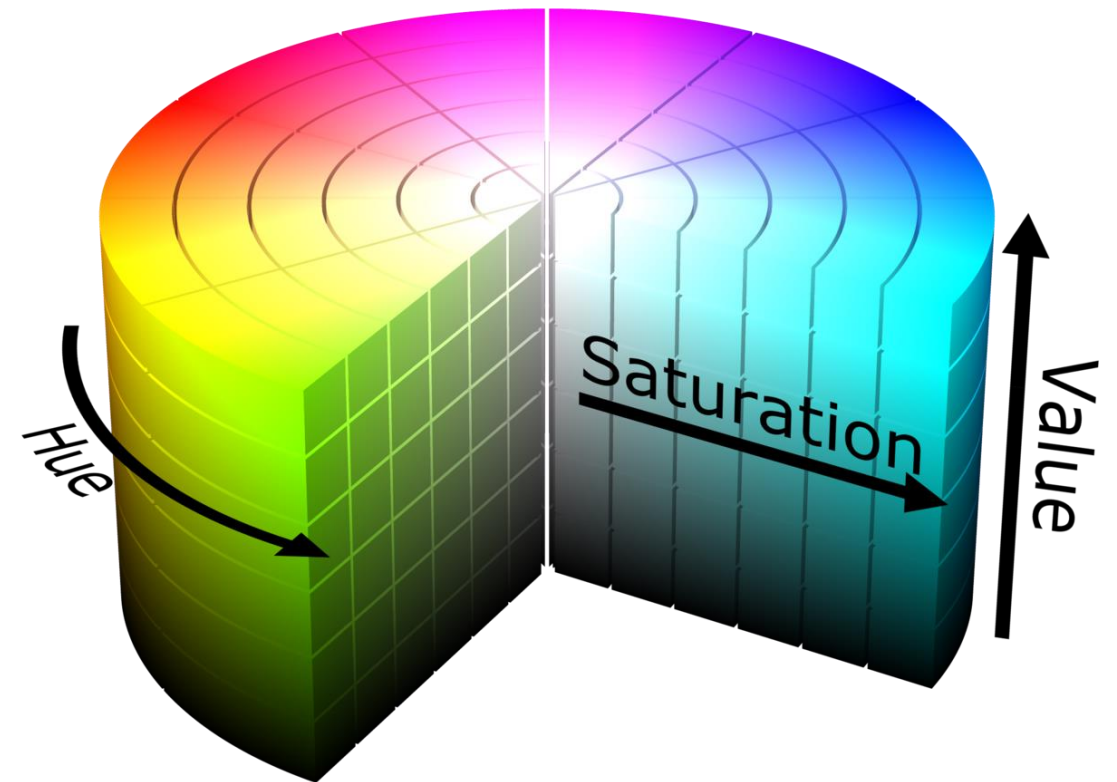
thixuanmay.le@unimelb.edu.au



RGB(red/green/blue)



HSV(Hue/Saturation/Value)



HSV(Hue/Saturation/Value)

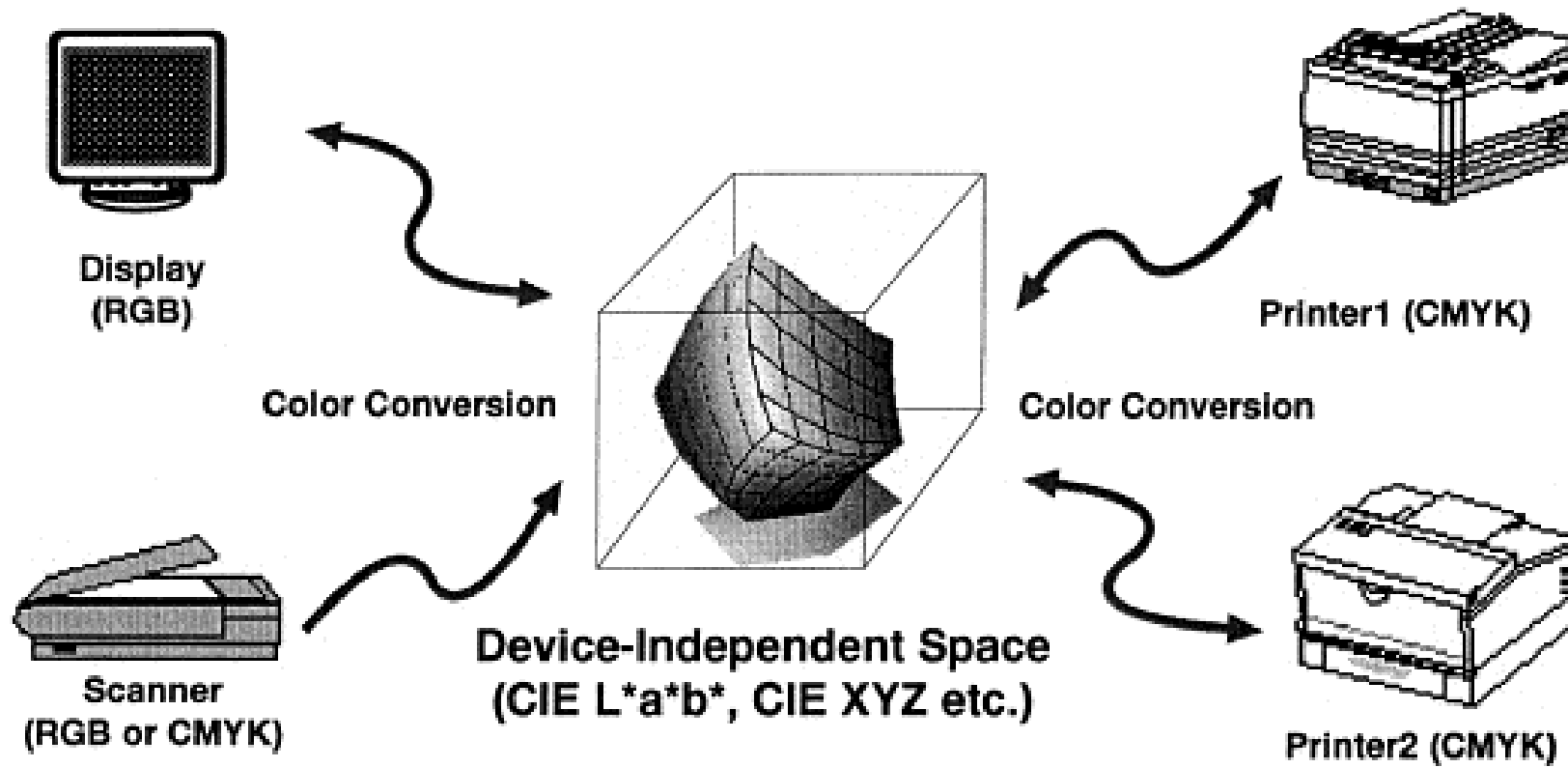


XYZ (device-independent)

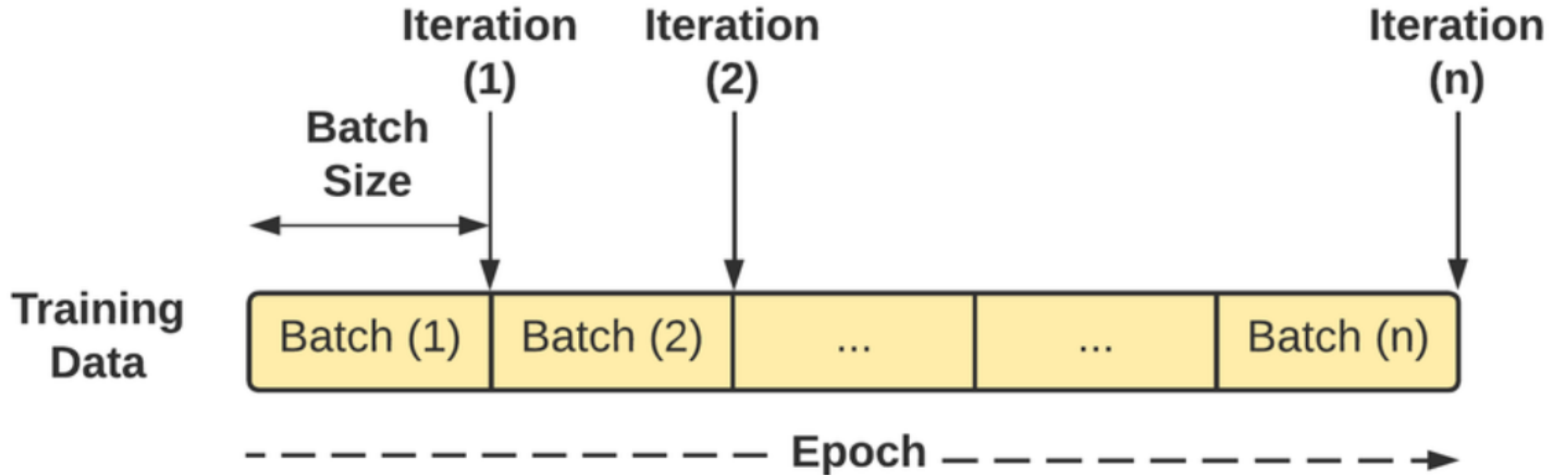
$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{pmatrix} \begin{pmatrix} R_{linear} \\ G_{linear} \\ B_{linear} \end{pmatrix}$$

- ❑ **X**: Represents a mixture of all wavelengths and corresponds roughly to the human eye's response to the red and green parts of the spectrum.
- ❑ **Y**: Represents the brightness or luminance of the color, closely matching the human eye's sensitivity to brightness.
- ❑ **Z**: Corresponds to the blue part of the spectrum.

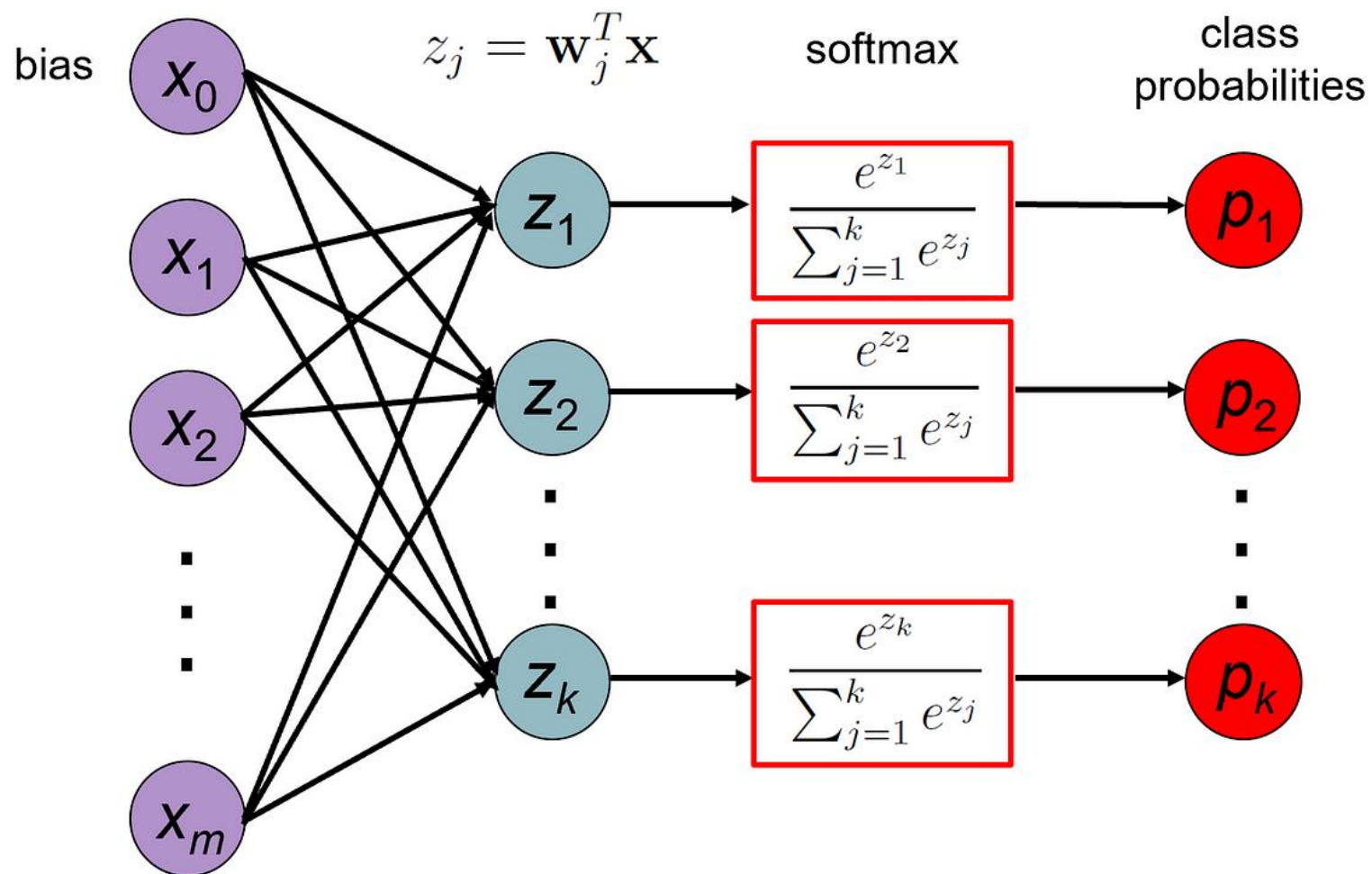
XYZ (device-independent)



Batch Size and Epoch

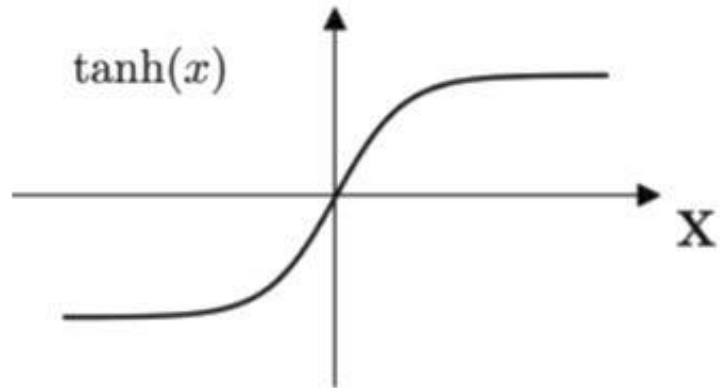


Softmax

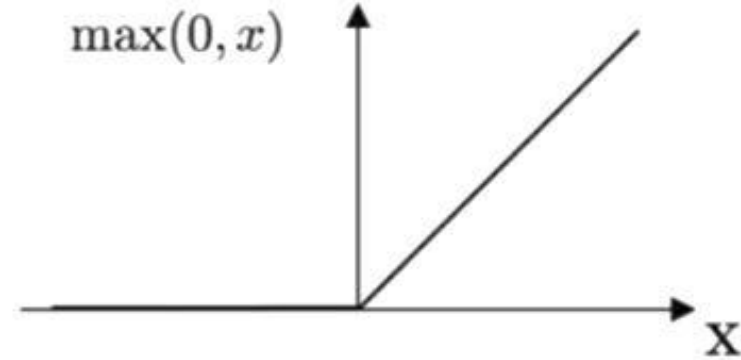


Other Activation Functions

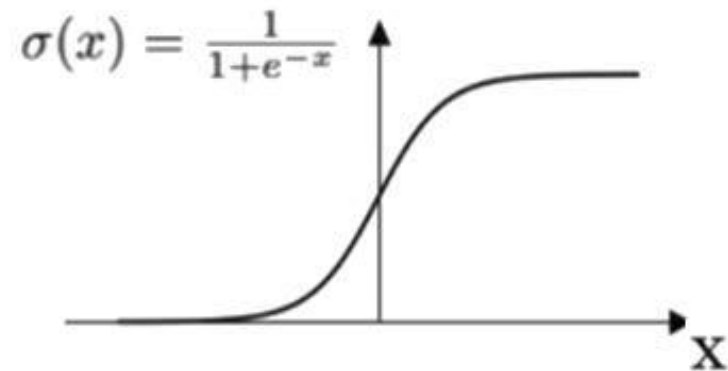
Tanh



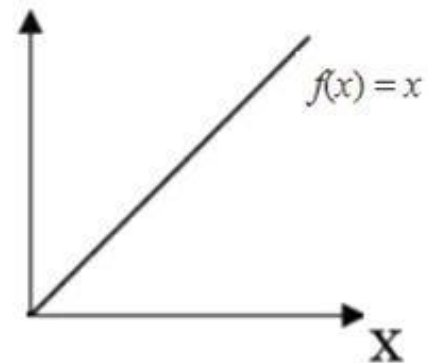
ReLU



Sigmoid



Linear



Conv2d

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

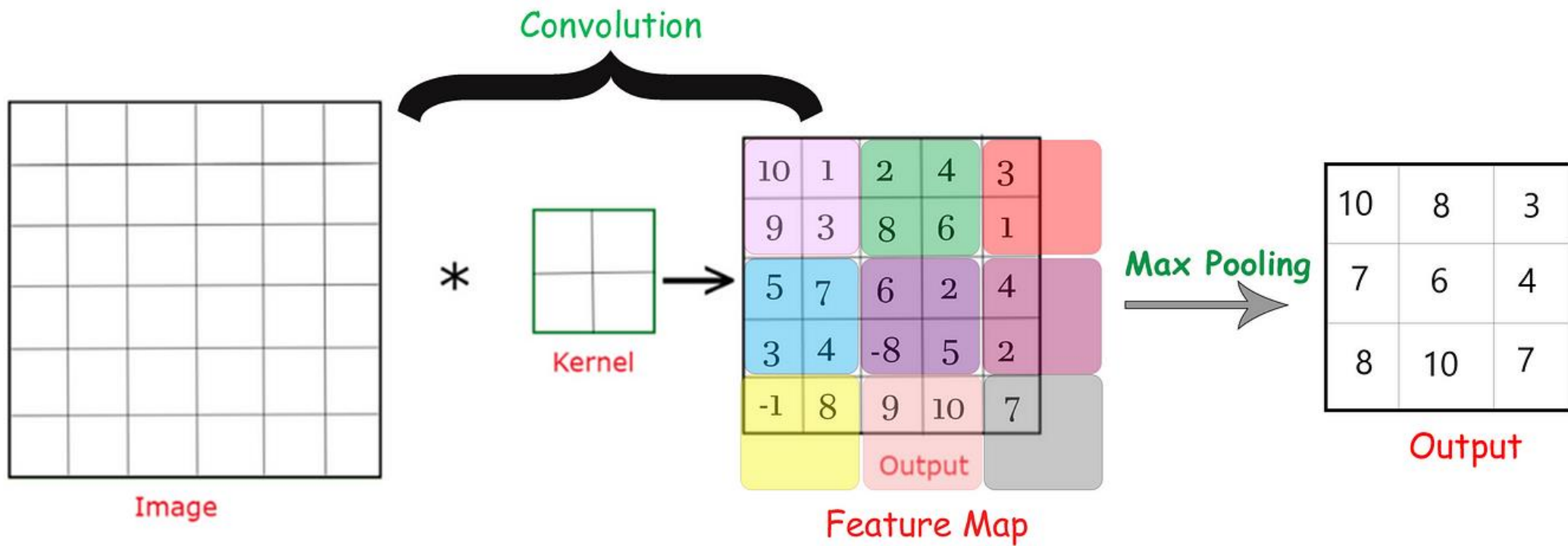
Conv2d

$$\text{Output Size} = \left(\frac{H - K_h + 2P}{S} + 1 \right) \times \left(\frac{W - K_w + 2P}{S} + 1 \right)$$

Where:

- H and W are the height and width of the input.
- K_h and K_w are the height and width of the kernel.
- P is the padding (0 in the case of 'valid' padding).
- S is the stride.

Max Pooling



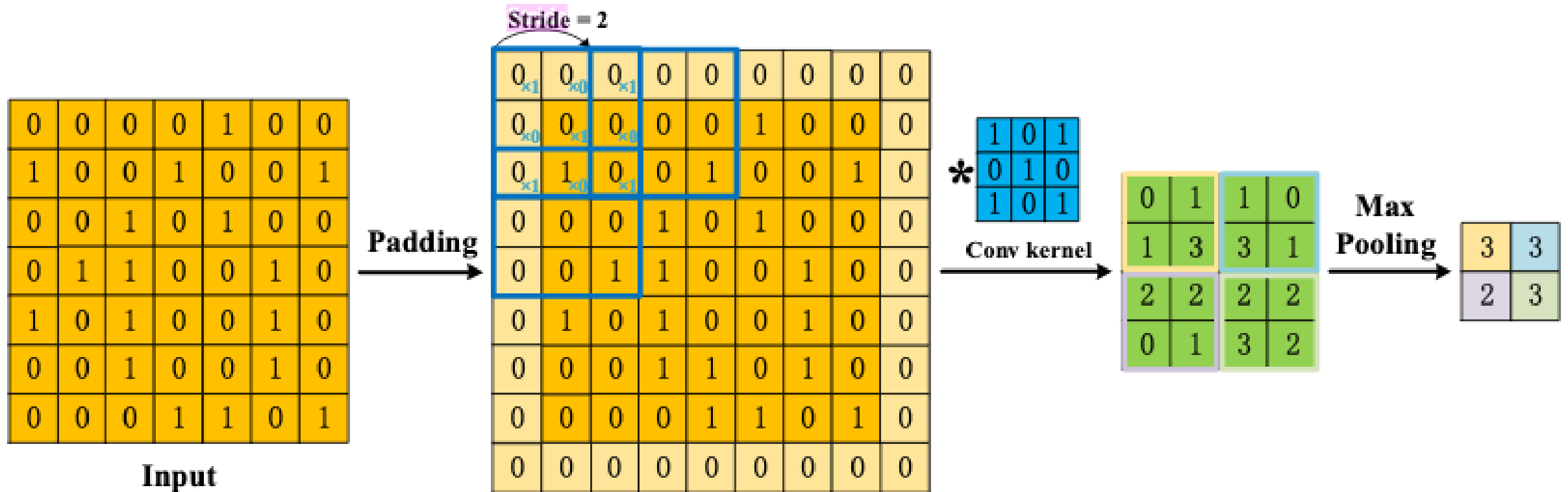
Max Pooling

$$\text{Output Size} = \left(\frac{H - F}{S} + 1 \right) \times \left(\frac{W - F}{S} + 1 \right) \times \text{Number of Channels}$$

Where:

- H and W are the height and width of the input.
- F is the size of the pooling filter.
- S is the stride.
- The number of channels remains unchanged.

Padding and Stride





Today's materials

Please access with
UniMelb account.





CNN Model

```
nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, stride=1, padding=0)
```

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

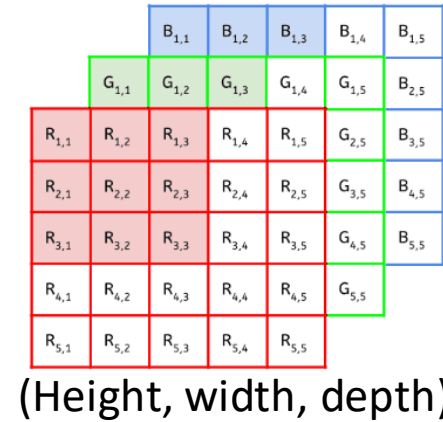
=

6		

$$\begin{aligned} &7 \times 1 + 4 \times 1 + 3 \times 1 + \\ &2 \times 0 + 5 \times 0 + 3 \times 0 + \\ &3 \times -1 + 3 \times -1 + 2 \times -1 \\ &= 6 \end{aligned}$$

- The kernel size denotes the dimensions of each convolution filter
- In most CNN architectures, it is commonly set to 3.

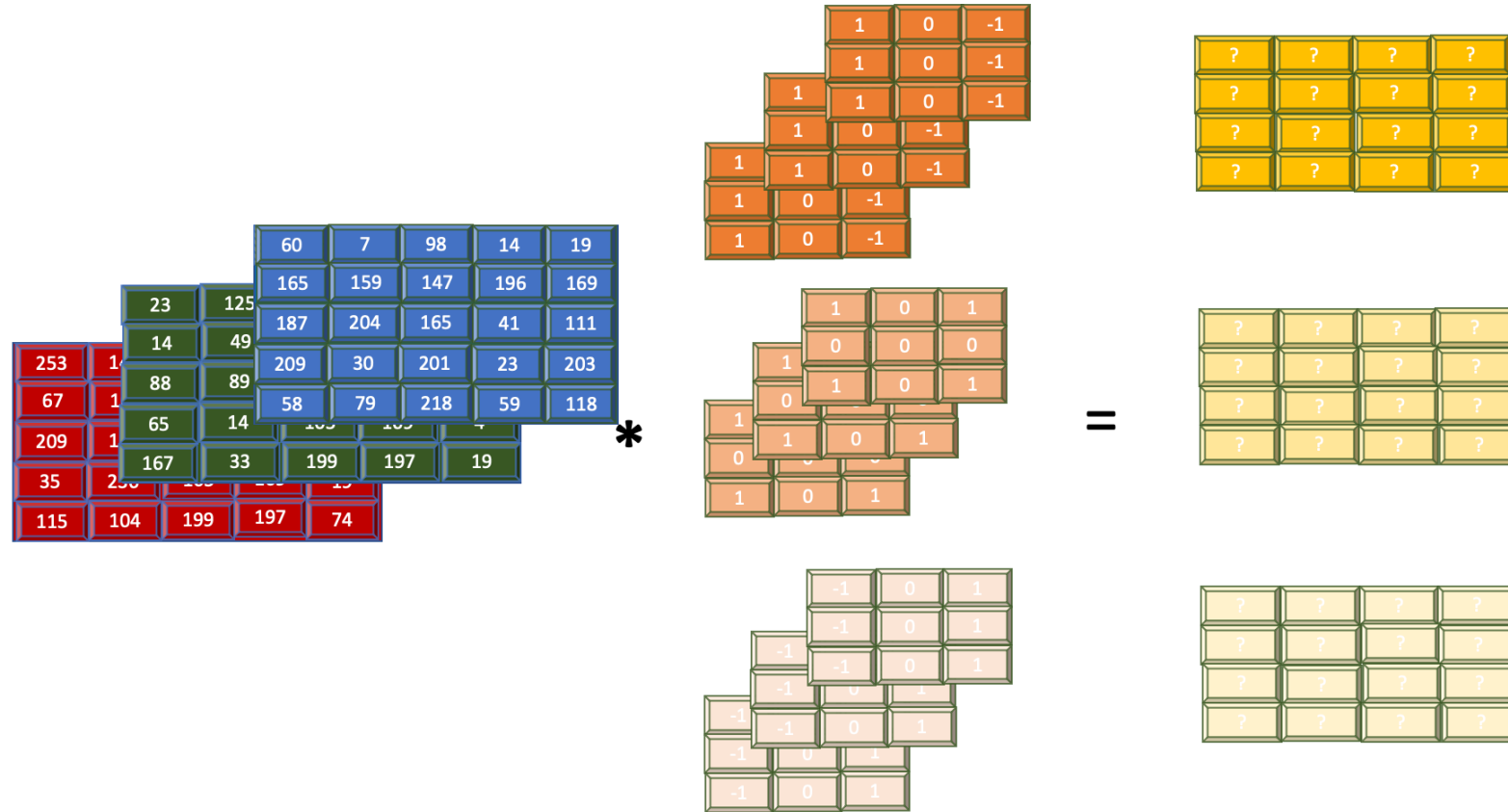
Example in PyTorch code



$$\Sigma =$$

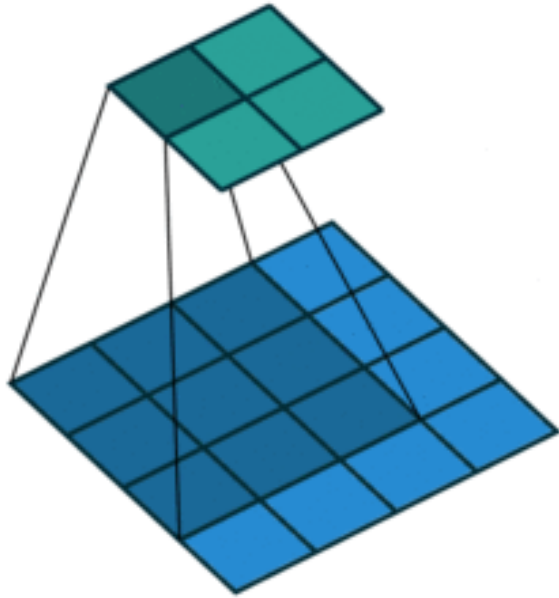
- The **in_channels** parameter must match the depth of the input (e.g., 3 for RGB images).
- The kernel depth matches the number of input channels, enabling it to process all channels simultaneously, for example, a 3×3×3 kernel for RGB images.

```
nn.Conv2d(in_channels=3, out_channels=3, kernel_size=3, stride=1, padding=0)
```

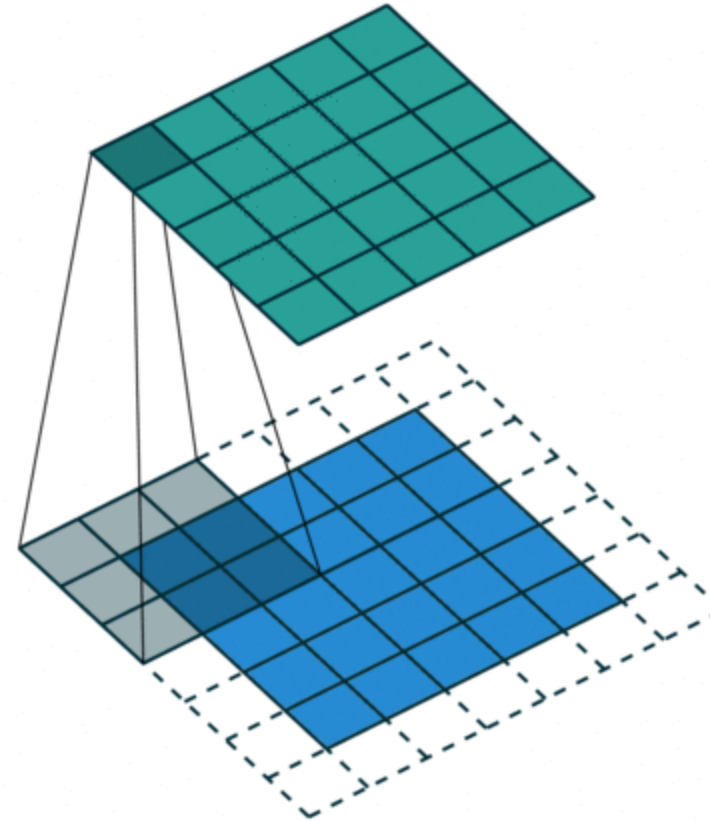


- **out_channels** is the number of filters.
- Each filter (kernel) generates one 2D tensor of shape **(1, h, w)**, so the output of the CNN layer has the shape **(out_channels, h, w)**.

```
nn.Conv2d(in_channels=3, out_channels=1, kernel_size=3, padding=0, stride=1)
```



padding=0

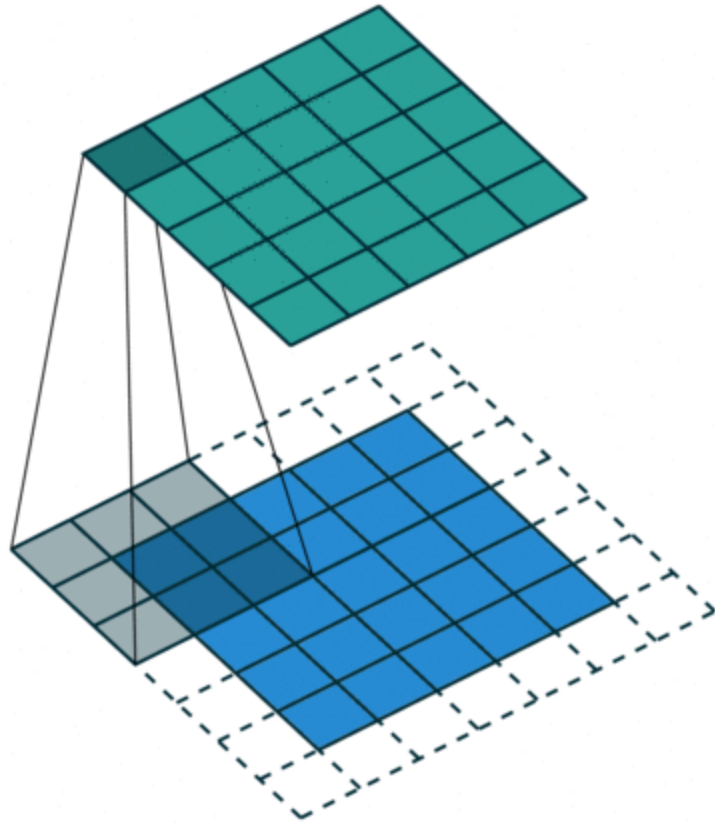


padding=1

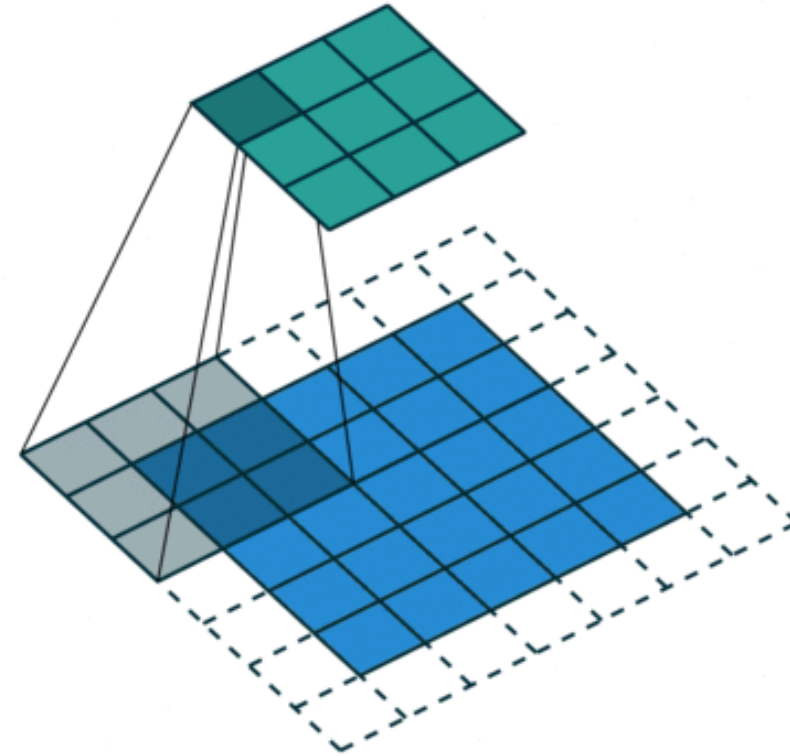
- With a kernel size of 3 and padding of 1, the output size remains the same as the input.
- In practical applications, the padding size is typically chosen as:

$$padding = \frac{kernel_size - 1}{2}$$

```
nn.Conv2d(in_channels=3, out_channels=1, kernel_size=3, padding=0, stride=2)
```



stride=1



stride=2

- A larger stride results in a smaller output size.
- In practical applications, the stride is typically set to 1 for convolution layers, but is often equal to the kernel size for pooling layers.

```
nn.MaxPool2d(kernel_size=2)
```

1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

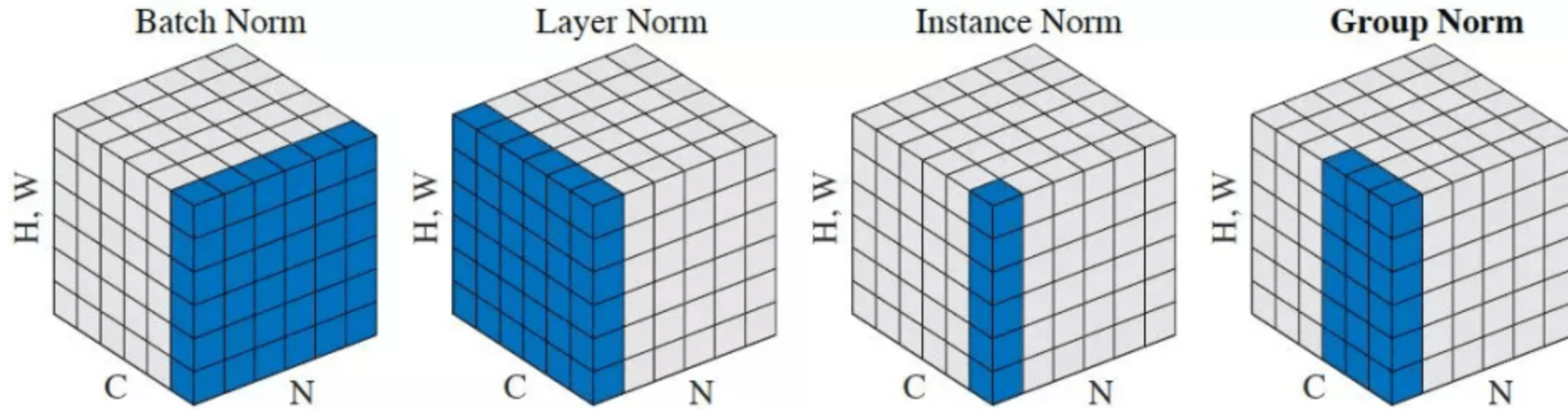
7	9
8	

- **MaxPool** designed to downsample the input.
- **MaxPool** takes the *maximum value within each pooling window.*

```
nn.AvgPool2d(kernel_size=2)
```

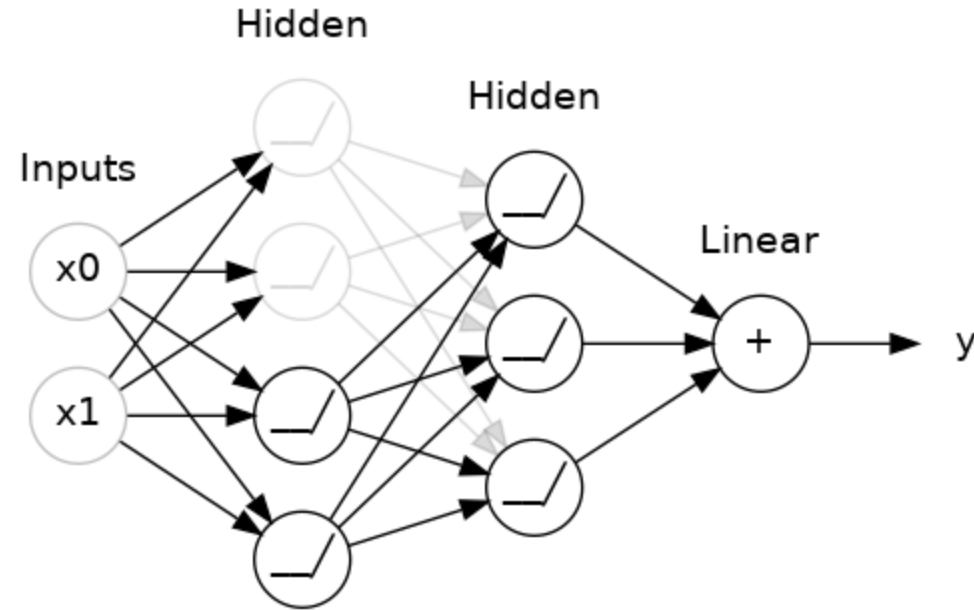
- **AvgPool** takes the *averaging value within each pooling window.*

```
nn.BatchNorm2d(num_features=32)
```



- BatchNorm is normalization between each tensor in one batch.
- **num_features** need to match the depth of the input

`nn.Dropout (p=0.2)`



- Dropout randomly turns off some neurons during training.
- This helps the model avoid relying too much on any one part and improves learning.